Tests

tests/Feature/App/CmcTest.php
tests/Feature/App/GdFTest.php
tests/Feature/App/GdRTest.php
tests/Feature/App/TdBTest.php
tests/Feature/Auth/AuthenticationTest.php
tests/Feature/Routes/AuthTest.php
tests/Feature/Routes/GuestTest.php

tests/Feature/App/CmcTest.php

```php
<?php

/**
 * Test sur la page Création / Modification de comptes
 */

use App\Enum\UserCategorieEnum;
use App\Models\User;
use Illuminate\Support\Facades\Hash;

/**
 * Tout les utilisateurs
 */
test('[UPDATE MDP] : Les utilisateurs peuvent mettre à jour le mdp', function () {
    $user = User::factory()->create();

    $mdpUpdate = array (
        'password' => 'newPassword' ,
        'password_confirmation'=> 'newPassword',
    );

    $response = $this
         ->actingAs($user)
         ->post(route('CMC.update'), $mdpUpdate);

    $this->assertTrue(Hash::check('newPassword', $user->password));
    $this->assertFalse(Hash::check('oldpassword', $user->password));

    $response->assertStatus(302);
});

/**
 * Administrateur
 */
test('[AJOUTER COMPTE] : Les administrateurs peuvent ajouter un compte', function ()
{
    $categorie = UserCategorieEnum::cases()[rand(0,2)]->value;
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
>create();

    $compteAdd = array (
        'prenom' => 'Test' ,
        'nom'=> 'Modifier',
```

```
        'email' => 'FakeEmailUpdate@test.com',
        'categorie' => $categorie,
        'password' => 'newPassword' ,
        'password_confirmation'=> 'newPassword',
    );

    $response = $this
         ->actingAs($admin)
         ->post(route('CMC.addUser'), $compteAdd);

    // Résultat
    $compte = User::where('prenom', $compteAdd['prenom'])->first();
    $this->assertEquals('Test', $compte->prenom);
    $this->assertEquals('Modifier', $compte->nom);
    $this->assertEquals('FakeEmailUpdate@test.com', $compte->email);
    $this->assertEquals($categorie, $compte->categorie->value);
    $this->assertTrue(Hash::check('newPassword', $compte->password));
    $this->assertFalse(Hash::check('oldpassword', $compte->password));

    $response->assertStatus(302);
});

test('[MODIFIER COMPTE] : Les administrateurs peuvent modifier un compte', function
() {
    $categorie = UserCategorieEnum::cases()[rand(0,2)]->value;
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
>create();
    $compte = User::factory()->state(['categorie' => $categorie])->create();

    $compteUpdate = array (
        'prenom' => 'Test' ,
        'nom'=> 'Modifier',
        'email' => 'FakeEmailUpdate@test.com',
        'categorie' => $categorie,
        'password' => 'newPassword' ,
        'password_confirmation'=> 'newPassword',
    );

    $response = $this
         ->actingAs($admin)
         ->post(route('CMC.updateUser', $compte->id), $compteUpdate);

    // Résultat
    $compte = User::find($compte->id);
    $this->assertEquals('Test', $compte->prenom);
    $this->assertEquals('Modifier', $compte->nom);
    $this->assertEquals('FakeEmailUpdate@test.com', $compte->email);
    $this->assertEquals($categorie, $compte->categorie->value);
    $this->assertTrue(Hash::check('newPassword', $compte->password));
    $this->assertFalse(Hash::check('oldpassword', $compte->password));

    $response->assertStatus(302);
});

test('[SUPPRIMER COMPTE] : Les administrateurs peuvent supprimer un compte', function
() {
    $categorie = UserCategorieEnum::cases()[rand(0,2)]->value;
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
```

```
>create();
    $compte = User::factory()->state(['categorie' => $categorie])->create();

    $response = $this
         ->actingAs($admin)
         ->post(route('CMC.delete', $compte->id));

    // Résultat
    $this->assertDatabaseMissing('users', ['id' => $compte->id]);
    $response->assertStatus(302);
});

test('[UPDATE INFO PERSONNEL] : Les administrateur peuvent mettre à jour leurs
informations personnelles', function () {
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
>create();

    $infoPerso = array (
        'prenom' => 'Test' ,
        'nom'=> 'Modifier',
        'email' => 'FakeEmailUpdate@test.com'
    );

    $response = $this
         ->actingAs($admin)
         ->post(route('CMC.updateInfoPerso'), $infoPerso);

    // Résultat
    $compte = User::find($admin->id);
    $this->assertEquals('Test', $compte->prenom);
    $this->assertEquals('Modifier', $compte->nom);
    $this->assertEquals('FakeEmailUpdate@test.com', $compte->email);

    $response->assertStatus(302);
});
```

tests/Feature/App/GdFTest.php

```
<?php

use App\Models\User;
use App\Enum\VehiculeTypeEnum;
use App\Enum\UserCategorieEnum;
use App\Enum\VehiculeStatutEnum;
use Illuminate\Http\UploadedFile;
use App\Enum\VehiculeTransmissionEnum;
use App\Models\Marque;
use App\Models\Modele;
use App\Models\Vehicule;
use Illuminate\Support\Facades\Storage;

/**
 * Gestionnaire
 */
```

```php
test('[AJOUTER VEHICULE] : Les gestionnaires peuvent ajouter un véhicule', function
() {

    // Variables importante
    $gestionnaire = User::factory()->state(['categorie' =>
UserCategorieEnum::GESTION])->create();
    $type = VehiculeTypeEnum::cases()[rand(0,3)]->value;
    $statut = VehiculeStatutEnum::cases()[rand(0,2)]->value;
    $transmission = VehiculeTransmissionEnum::cases()[rand(0,1)]->value;
    Storage::fake('voitureImg');

    // On crée un faux formulaire
    $vehiculeAdd = array (
        'vehicule_image' => UploadedFile::fake()->image('voitureImg.jpg') ,
        'id'=> 'CH 999 999',
        'tarifj' => '11.11',
        'marque' => 'Test',
        'modele' => 'T2' ,
        'type'=> $type,
        'statut' => $statut,
        'transmission' => $transmission
    );

    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdF.addCar'), $vehiculeAdd);

    /**
     *  Résultat
     */
    // On récupère le véhicule crée
    $vehicule = Vehicule::where('id', $vehiculeAdd['id'])->first();

    // On s'assure que l'image est bien enregistrer au bon endroit
    Storage::disk('public')->assertExists($vehicule->image);

    // Résultat
    $this->assertEquals('CH 999 999', $vehicule->id);
    $this->assertEquals('11.11', $vehicule->tarif_journalier);
    $this->assertEquals('Test', $vehicule->marque->libelle);
    $this->assertEquals('T2', $vehicule->modele->libelle);
    $this->assertEquals($type, $vehicule->type);
    $this->assertEquals($statut, $vehicule->statut);
    $this->assertEquals($transmission, $vehicule->transmission);
    $response->assertStatus(302);

    // On supprimer l'image test
    Storage::disk('public')->delete($vehicule->image);
});

test('[SUPPRIMER VEHICULE] : Les gestionnaires peuvent supprimer un véhicule',
function () {
    $gestionnaire = User::factory()->state(['categorie' =>
UserCategorieEnum::GESTION])->create();
    Marque::factory()->create();
    Modele::factory()->create();
    $vehicule = Vehicule::factory()->create();
```

```
    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdF.deleteCar', $vehicule->id));

    // Résultat

    // On regarde si l'image est bien supprimer
    Storage::disk('public')->assertMissing($vehicule->image);
    $this->assertDatabaseMissing('vehicules', ['id' => $vehicule->id]);
    $response->assertStatus(302);
});

test('[UPDATE/STATUT VEHICULE] : Les gestionnaires peuvent mettre à jour le statut
d\'un véhicule', function () {

    // Variables importante
    $gestionnaire = User::factory()->state(['categorie' =>
UserCategorieEnum::GESTION])->create();
    Marque::factory()->create();
    Modele::factory()->create();
    $statut = VehiculeStatutEnum::cases()[rand(0,2)]->value;
    $vehicule = Vehicule::factory()->create();

    // Prepare la requete
    $updateVehicule = array (
        'id_vehicule' => $vehicule->id,
        'statut' => $statut,
    );


    // On envoie la requete par le gestionnaire
    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdF.updateStatut', $updateVehicule));

    // Résultat
    // On recupere le vehicule mis a jours normalement
    $vehiculeU = Vehicule::find($vehicule->id);
    // On verifie que le statut est bien a jour
    $this->assertEquals($statut, $vehiculeU->statut);
    $response->assertJson(['success' => 'Statut mis a jour!']);
    $response->assertStatus(200);

    // On supprimer l'image test
    Storage::disk('public')->delete($vehiculeU->image);
});
```

tests/Feature/App/GdRTest.php

```
<?php

use Carbon\Carbon;
use App\Models\User;
```

```php
use App\Models\Marque;
use App\Models\Modele;
use App\Models\Location;
use App\Models\Vehicule;
use App\Enum\UserCategorieEnum;
use App\Enum\VehiculeStatutEnum;

test('[RESERVATION VEHICULE] : Les gestionnaires / clients peuvent réserver un véhicules.', function () {

    // Variables importants
    $gestionnaire = User::factory()->state(['categorie' => UserCategorieEnum::GESTION])->create();
    $clientGestion = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT, 'montant_des_locations' => 0])->create();
    Marque::factory()->create();
    Modele::factory()->create();
    $vehiculeGestion = Vehicule::factory()->state(['statut' => VehiculeStatutEnum::DISPO, 'tarif_journalier' => '10.00'])->create();
    $dureeL = rand(1,200);

    // Calcule de la duree
    $currentTime = Carbon::now();
    $calculTime = now()->parse($currentTime)->addDays($dureeL);
    $dateDebut = date(date('Y-m-d',strtotime($currentTime)));
    $dateFin = date('Y-m-d',strtotime($calculTime));

    // Calcul du montant du client
    # Variables importante
    $prixSansReduction = $dureeL*$vehiculeGestion->tarif_journalier;
    $pourcentageSemaine = 10/100;
    $pourcentageMois    = 20/100;

    // On regarde si une reduction s'applique avant le prix final
    if ($dureeL >= 7)
    {
        if ($dureeL >= 30)
        {
            $prixLocation = (($prixSansReduction) - (($prixSansReduction) * ($pourcentageMois)));
        }
        else {
        $prixLocation = (($prixSansReduction) - (($prixSansReduction) * ($pourcentageSemaine)));
        }
    }
    else {
        $prixLocation = $prixSansReduction;
    }

    $bookCarGestion = array (
        'duree_location'    => $dureeL,
        'client_id'         => $clientGestion->id,
        'vehicule_id'       => $vehiculeGestion->id,
    );

    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdR.bookCar'), $bookCarGestion);
```

```
    // On recupere les donnees a jours
    $vehiculeGestionBook = Vehicule::find($vehiculeGestion->id);
    $clientGestionM = User::find($clientGestion->id);
    $location = Location::where('vehicule_id', $vehiculeGestionBook->id)->first();

    // Resultat
    $this->assertEquals($clientGestionM->id, $location->client_id);
    $this->assertEquals($vehiculeGestionBook->id, $location->vehicule_id);
    $this->assertEquals($dateDebut, $location->date_debut);
    $this->assertEquals($dateFin, $location->date_fin);
    $this->assertEquals(VehiculeStatutEnum::LOUER->value, $vehiculeGestionBook->statut);
    $this->assertEquals($prixLocation, $clientGestionM->montant_des_locations);
    $response->assertStatus(302);

    // Le client effectue sa reservation
    $client = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT, 'montant_des_locations' => 0])->create();
    $vehiculeClient = Vehicule::factory()->state(['statut' => VehiculeStatutEnum::DISPO, 'tarif_journalier' => '10.00'])->create();

    $bookCarClient = array (
        'duree_location'    => $dureeL,
        'vehicule_id'       => $vehiculeClient->id,
    );

    $response = $this
        ->actingAs($client)
        ->post(route('GdR.bookCar'), $bookCarClient);

        // On recupere les donnees a jours
        $vehiculeClientBook = Vehicule::find($vehiculeClient->id);
        $clientM = User::find($client->id);
        $location = Location::where('vehicule_id', $vehiculeClientBook->id)->first();

        // Resultat
        $this->assertEquals($clientM->id, $location->client_id);
        $this->assertEquals($vehiculeClientBook->id, $location->vehicule_id);
        $this->assertEquals($dateDebut, $location->date_debut);
        $this->assertEquals($dateFin, $location->date_fin);
        $this->assertEquals(VehiculeStatutEnum::LOUER->value, $vehiculeClientBook->statut);
        $this->assertEquals($prixLocation, $clientM->montant_des_locations);
        $response->assertStatus(302);
});
```

tests/Feature/App/TdBTest.php

```
<?php

use Carbon\Carbon;
use App\Models\User;
use App\Models\Marque;
```

```php
use App\Models\Modele;
use App\Models\Location;
use App\Models\Vehicule;
use App\Enum\UserCategorieEnum;
use App\Enum\VehiculeStatutEnum;
use Illuminate\Support\Facades\Storage;

test('[UPDATE/STATUT VEHICULE] : Les gestionnaires peuvent changer le statut d\'un vehicule une fois la fin de la location et non avant', function () {

    // Variables importante
    $gestionnaire = User::factory()->state(['categorie' => UserCategorieEnum::GESTION])->create();
    $client = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT])->create();
    Marque::factory()->create();
    Modele::factory()->create();
    $statut = VehiculeStatutEnum::cases()[rand(0,2)]->value;
    $vehicule = Vehicule::factory()->state(['statut' => VehiculeStatutEnum::LOUER])->create();
    $dureeL = rand(1,200);

    // Faire une date de location echue et non echue
    $currentTime = Carbon::now();
    $calculTime = now()->parse($currentTime)->addDays($dureeL);
    $finDeLaLocation = date('Y-m-d',strtotime($calculTime->addDay()));
    $avantFinDeLaLocation = date('Y-m-d',strtotime($calculTime->subDays(2)));

    // On doit faire en sorte que la date actuelle est superieur d'un jour que la location "Session('currentDate')"
    // Prepare la requete
    $changeCurrentDate = array (
        'currentDate' => $finDeLaLocation
    );

    $response = $this
        ->actingAs($gestionnaire)
        ->post(route('TdB.changeCurrentDate', $changeCurrentDate));

    // On doit cree la location de la voiture
    $bookCar = array(
        'duree_location'    => $dureeL,
        'client_id'         => $client->id,
        'vehicule_id'       => $vehicule->id,
    );

    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdR.bookCar'), $bookCar);

    $location = Location::where('vehicule_id', $vehicule->id)->first();

    // On tester le changement de statut post date
    // Prepare la requete
    $updateVehicule = array (
        'id_vehicule' => $vehicule->id,
        'statut' => $statut,
    );
```

```php
    // On envoie la requete par le gestionnaire
    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('TdB.updateStatut', $updateVehicule));

    // Résultat
    // On recupere le vehicule mis a jours normalement
    $vehiculeU = Vehicule::find($vehicule->id);
    // On verifie que le statut est bien a jour
    $this->assertEquals($statut, $vehiculeU->statut);
    // On s'assure que le contrat est supprimer
    $this->assertDatabaseMissing('locations', ['id' => $location->id]);
    // On s'assure de la code HTTP est bon
    $response->assertStatus(200);

    /**
     * Ne peut pas changer avant la fin du contrat
     */

    // On doit faire en sorte que la date actuelle est superieur d'un jour que la
location "Session('currentDate')"
    // Prepare la requete
    $changeCurrentDate = array (
        'currentDate' => $avantFinDeLaLocation
    );

    $response = $this
        ->actingAs($gestionnaire)
        ->post(route('TdB.changeCurrentDate', $changeCurrentDate));

    // On doit cree la location de la voiture
    $bookCar = array(
        'duree_location'    => $dureeL,
        'client_id'         => $client->id,
        'vehicule_id'       => $vehicule->id,
    );

    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('GdR.bookCar'), $bookCar);

    $location = Location::where('vehicule_id', $vehicule->id)->first();

    // On tester le changement de statut avant date de fin de location
    // Prepare la requete
    $updateVehicule = array (
        'id_vehicule' => $vehicule->id,
        'statut' => $statut,
    );

    // On envoie la requete par le gestionnaire
    $response = $this
         ->actingAs($gestionnaire)
         ->post(route('TdB.updateStatut', $updateVehicule));

    // Résultat
```

```
    // On recupere le vehicule mis a jours normalement
    $vehiculeU2 = Vehicule::find($vehicule->id);
    // On verifie que le statut est bien a jour
    $this->assertEquals(VehiculeStatutEnum::LOUER->value, $vehiculeU2->statut);
    // On s'assure que le contrat est supprimer
    $this->assertEquals($vehiculeU2->id, $location->vehicule_id);
    // On s'assure de la code HTTP est bon
    $response->assertStatus(200);

    // On supprimer l'image test
    Storage::disk('public')->delete($vehiculeU->image);
});
```

tests/Feature/Auth/AuthenticationTest.php

```
<?php

use App\Models\User;
use App\Providers\RouteServiceProvider;

/**
 * Test sur les connexions dans la page "LOGIN"
 */

test('[LOGIN] : Les utilisateurs peuvent se connecter à l\'application', function ()
{
    $user = User::factory()->create();

    $response = $this->post('/login', [
        'email' => $user->email,
        'password' => 'password',
    ]);

    $this->assertAuthenticated();
    $response->assertRedirect(RouteServiceProvider::HOME);
});

test('[LOGIN] : Les utilisateurs ne peuvent pas se connecter avec un mauvais mot de
passe', function () {
    $user = User::factory()->create();

    $this->post('/login', [
        'email' => $user->email,
        'password' => 'wrong-password',
    ]);

    $this->assertGuest();
});
```

tests/Feature/Routes/AuthTest.php

```php
<?php

use App\Enum\UserCategorieEnum;
use App\Models\User;

/**
 * Il s'agit des tests sur l'acces des pages (Url) uniquement si nous sommes
connecter
 */

test('[CREATION/MODIFICATION - COMPTES] : Accès possible.', function () {
    $user = User::factory()->create();

    $response = $this->actingAs($user)->get(route('CMC'));

    $response->assertStatus(200);
});

test('[GESTION - RESERVATION] : Accès possible.', function () {
    // Categorie d'utilisateur
    $client = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT])-
>create();
    $gestionnaire = User::factory()->state(['categorie' =>
UserCategorieEnum::GESTION])->create();
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
>create();

    /**
     * Resultat attendu
     */

    //Client
    $response = $this->actingAs($client)->get(route('GdR'));
    $response->assertStatus(200);

    // Gestionnaire
    $response = $this->actingAs($gestionnaire)->get(route('GdR'));
    $response->assertStatus(200);

    // Administrateur
    $response = $this->actingAs($admin)->get(route('GdR'));
    $response->assertStatus(302);
});

test('[GESTION - FLOTTE] : Accès possible.', function () {
   // Categorie d'utilisateur
   $client = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT])-
>create();
   $gestionnaire = User::factory()->state(['categorie' =>
UserCategorieEnum::GESTION])->create();
   $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])-
>create();

   /**
    * Resultat attendu
    */

   //Client
   $response = $this->actingAs($client)->get(route('GdF'));
```

```
    $response->assertStatus(302);

    // Gestionnaire
    $response = $this->actingAs($gestionnaire)->get(route('GdF'));
    $response->assertStatus(200);

    // Administrateur
    $response = $this->actingAs($admin)->get(route('GdF'));
    $response->assertStatus(200);
});

test('[TABLEAU DE BORD] : Accès possible.', function () {
     // Categorie d'utilisateur
    $client = User::factory()->state(['categorie' => UserCategorieEnum::CLIENT])->create();
    $gestionnaire = User::factory()->state(['categorie' => UserCategorieEnum::GESTION])->create();
    $admin = User::factory()->state(['categorie' => UserCategorieEnum::ADMIN])->create();

    /**
     * Resultat attendu
     */

    //Client
    $response = $this->actingAs($client)->get(route('TdB'));
    $response->assertStatus(302);

    // Gestionnaire
    $response = $this->actingAs($gestionnaire)->get(route('TdB'));
    $response->assertStatus(200);

    // Administrateur
    $response = $this->actingAs($admin)->get(route('TdB'));
    $response->assertStatus(200);
});
```

tests/Feature/Routes/GuestTest.php

```
<?php
/**
 * Il s'agit des tests sur l'acces des pages (URL) si nous ne sommes pas connecter
 */

/**
 * Route pour la page de connexion
 */
test('[CONNEXION] : Accès possible.', function () {
    $response = $this->get('/login');
    $response->assertStatus(200);
});

/**
```

```php
 * Route pour le changement de langue du site
 */
test('[CHANGEMENT DE LANGUE] : Accès possible.', function () {
    $response = $this->get('/lang/?lang=');
    $response->assertStatus(302);
});
```